

Projeto 01: Busca Adversarial em Jogos

Agente Inteligente para Othello (Reversi)

Bruno Amaral Salles (281129) • Rodrigo Banin Ferraz de Camargo (238257)

Caio Lima Albuquerque (288808) • João Pedro Bonatti (237575) • Ricardo Andrade Oliveira Bello (247326)

March 31, 2026

1 Introdução - Othello/Reversi

O jogo Othello (também chamado de Reversi) é um jogo de tabuleiro de estratégia para dois jogadores, jogado em um tabuleiro de 8×8 . Há sessenta e quatro peças de jogo idênticas chamadas discos, que são claras de um lado e escuras do outro. Os jogadores se revezam colocando peças no tabuleiro com a cor atribuída voltada para cima. Durante uma jogada, quaisquer peças da cor do adversário que estejam em linha reta (horizontal, vertical ou diagonal) e limitados pelo disco acabado de colocar e outro disco da cor do jogador atual serão revertidos (viradas) à cor do jogador atual. O jogador com mais discos no tabuleiro, ao final da partida, vence. Se ambos tiverem o mesmo número de discos, a partida termina em empate.

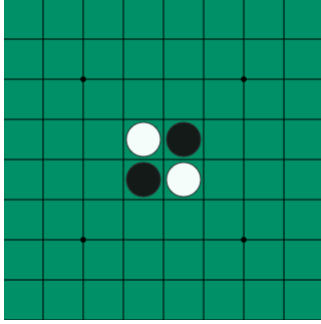


Figure 1: Estado inicial do tabuleiro

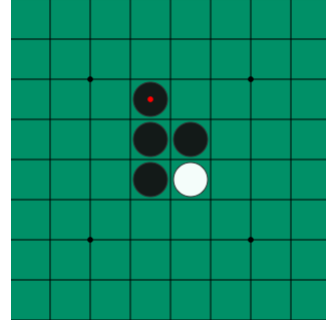


Figure 2: Primeiro movimento das peças pretas

Uma vez que uma peça é colocada em um canto ela se torna "estável", isto é, não pode ser virada novamente. Por isso, a principal estratégia de jogo do Othello passa por garantir o controle sobre os cantos do tabuleiro e, a partir daí, construir mais peças estáveis adjacentes. Esse projeto explora essa estratégia, assim como a de restrição de mobilidade do oponente e discos de fronteiras. Em particular, as características que permitem aplicar os algoritmos de busca descritos nesse relatório são que o jogo Othello é determinístico, de soma zero e de observabilidade total.

2 Modelagem do Problema

2.1 Definição Formal

A fim de elaborar algoritmos para execução do jogo, é necessário definir formalmente o jogo como uma tupla do tipo (S, A, T, U)

- **Estado (S):** Um estado $s \in \mathcal{S}$ é definido, em um dado momento do jogo, como uma matriz 8×8 que representa o tabuleiro. Cada posição $B_{i,j} \in \{\emptyset, \text{Preto}, \text{Branco}\}$, e $p \in \{\text{Preto}, \text{Branco}\}$ indica de quem é a vez de jogar. O estado inicial $\mathcal{S}_0$ contém 4 peças no centro do tabuleiro (duas brancas e duas pretas, cruzadas) e $p = \text{Preto}$.
- **Ações (A):** Seja o conjunto A o conjunto de ações possíveis a serem executadas pelo jogador p em um dado estado s do jogo. A ação $a \in A$ tal que $a = (i, j)$ é válida (e, portanto, pode ser executada) se:

1. $B_{i,j} = \emptyset$.
 2. Colocar a peça na posição (i, j) forma uma linha contínua (horizontal, vertical ou diagonal) de peças inimigas terminada por uma peça amiga.
- **Transição (T):** A função de transição aplica o movimento no tabuleiro, alterando a cor de todas as peças flanqueadas nas 8 direções e passando o turno para o próximo jogador. No caso da ação ser nula, a matriz do tabuleiro se mantém sem alterações
 - **Utilidade(U):** Seja $U(s)$ a função de utilidade para um dado estado s . U depende da Heurística que está sendo utilizada para avaliar o tabuleiro:
 1. **Controle de Borda:** Avalia a posse dos quatro cantos do tabuleiro. Seja $C = \{(1, 1), (1, 8), (8, 1), (8, 8)\}$ o conjunto de coordenadas dos cantos na matriz. A função recompensa ter a posse destes espaços e penaliza a posse do adversário:

$$U_{corner}(s, p) = \sum_{(i,j) \in C} I(B_{i,j} = p) - \sum_{(i,j) \in C} I(B_{i,j} = \neg p)$$

2. **Mobilidade:** A utilidade é calculada comparando a diferença de cardinalidade dos conjuntos de ações válidas disponíveis para ambos os jogadores em um dado estado, isto é, maximiza a diferença entre ações válidas do jogador p em relação as ações válidas do Oponente:

$$U_{mobility}(s, p) = |A(p)| - |A(\neg p)|$$

3. **Fronteira:** Denomina-se disco de fronteira todo disco posicionado nas 8 direções adjacentes a pelo menos uma célula vazia. Seja $\mathcal{F}(s, x)$ a função que quantifica o total de discos de fronteira do jogador x . A utilidade premia ter menos discos na fronteira que o inimigo:

$$U_{frontier}(s, p) = \mathcal{F}(s, \neg p) - \mathcal{F}(s, p)$$

2.2 Representação de Estado

A definição formal de estado apresentada na seção anterior foi convertida em uma estrutura de dados que representa o tabuleiro, a qual nomeamos **GameState**, que armazena as propriedades do tabuleiro em um dado momento. O tabuleiro é representado por uma Matriz bidimensional B no formato 8×8 . Cada elemento da matriz pertence ao conjunto $B_{i,j} \in \{\text{Vazio}, \text{Preto}, \text{Branco}\}$ seguindo a seguinte correspondência:

$$\bullet \text{ Vazio} = 0 \quad \bullet \text{ Preto} = 1 \quad \bullet \text{ Branco} = -1$$

Para um dado estado, armazena-se também qual o jogador p que irá realizar a próxima jogada. Feita uma jogada, a representação do turno é invertida seguindo a regra matemática direta $p = p \times (-1)$.

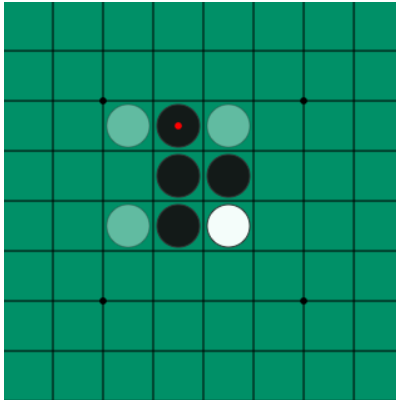


Figure 3: Representação do estado $S_t = (B, \text{Branco})$. As peças brancas translúcidas representam as possíveis jogadas do jogador branco.

2.3 Geração de Ações

A geração de ações $\mathcal{A}(s)$ percorre as células vazias e utiliza buscas radiais (em 8 direções) para validar jogadas válidas. Durante a transição de um estado para os filhos, a matriz original é clonada para evitar modificações ao estado real do jogo, aplica-se as jogadas, e o turno passado para o outro jogador em uma expansão da árvore de possibilidades. Em caso de ações vazias para ambos os jogadores, o nó é considerado um estado terminal

Algoritmo 1: Geração de Jogadas Válidas para Othello

```
1 Função Gerar_Ações(tabuleiro, jogador):  
2   ações_validas  $\leftarrow \emptyset$ ;  
3   for  $i \leftarrow 0$  to 7 do  
4     for  $j \leftarrow 0$  to 7 do  
5       if tabuleiro[ $i$ ][ $j$ ] = VAZIO then  
6         flanqueados  $\leftarrow$  Buscar_Flanqueamentos(tabuleiro,  $i, j$ , jogador);  
7         if flanqueados  $\neq \emptyset$  then  
8           Adicionar ( $i, j$ ) ao conjunto ações_validas;  
9         end  
10      end  
11    end  
12  end  
13  return ações_validas;  
14 fim
```

3 Algoritmos de Busca

Nesta seção descrevemos os dois algoritmos de busca adversarial implementados — Minimax e Alpha-Beta Pruning — bem como as técnicas complementares de *Iterative Deepening* com limite de tempo e ordenação de jogadas (*Move Ordering*).

3.1 Minimax

O algoritmo Minimax modela o jogo como uma árvore de decisão em que dois jogadores alternam turnos: o jogador MAX busca maximizar a utilidade do estado, enquanto o jogador MIN busca minimizá-la. A premissa é que o adversário joga de forma ótima, de modo que o agente escolhe o lance que garante o melhor resultado no pior cenário.

A implementação percorre recursivamente a árvore de jogo até uma profundidade limite d ou até atingir um estado terminal. Nos nós folha, a função heurística $U(s, p)$ é avaliada. Quando um jogador não possui jogadas válidas (passe forçado), o turno é transferido ao oponente e a recursão prossegue. A cada nó visitado, incrementa-se o contador de nós expandidos para fins de instrumentação.

Algoritmo 2: Minimax

```
1 Função Minimax(tabuleiro, jogador_atual, jogador_max, profundidade, heurística):
2   nós_expandidos  $\leftarrow$  nós_expandidos + 1;
3   if profundidade = 0 ou estado é terminal then
4     | return heurística(tabuleiro, jogador_max);
5   end
6   jogadas  $\leftarrow$  jogadas_válidas(tabuleiro, jogador_atual);
7   if jogadas =  $\emptyset$  ; // passe forçado
8     then
9       | return Minimax(tabuleiro, oponente(jogador_atual), jogador_max, profundidade - 1, heurística);
10    end
11    if jogador_atual = jogador_max then
12      | valor  $\leftarrow$   $-\infty$ ;
13      | foreach (jogada, filho)  $\in$  ordenar_filhos(jogadas) do
14        | valor  $\leftarrow$  max(valor, Minimax(filho, oponente(jogador_atual), jogador_max, profundidade - 1,
15        |   heurística));
16      | end
17    else
18      | valor  $\leftarrow$   $+\infty$ ;
19      | foreach (jogada, filho)  $\in$  ordenar_filhos(jogadas) do
20        | valor  $\leftarrow$  min(valor, Minimax(filho, oponente(jogador_atual), jogador_max, profundidade - 1,
21        |   heurística));
22      | end
23    end
24    return valor;
25 fim
```

A complexidade temporal do Minimax é $O(b^d)$, onde b é o fator de ramificação médio e d a profundidade de busca. No Othello, $b \approx 10$ no meio do jogo, tornando buscas profundas computacionalmente caras sem otimizações adicionais.

3.2 Alpha-Beta Pruning

O algoritmo Alpha-Beta Pruning é uma otimização do Minimax que produz a mesma decisão final, porém elimina ramos da árvore que comprovadamente não podem influenciar o resultado. Para isso, mantém dois limites durante a busca:

- α : melhor valor já garantido pelo jogador MAX (limite inferior);
- β : melhor valor já garantido pelo jogador MIN (limite superior).

Sempre que $\beta \leq \alpha$ em um nó, os ramos restantes são podados, pois o jogador racional (MAX ou MIN) jamais permitirá que a busca alcance aquele estado.

Algoritmo 3: Alpha-Beta Pruning

```
1 Função AlphaBeta(tabuleiro, jogador_atual, jogador_max, profundidade,  $\alpha$ ,  $\beta$ , heurística):
2   nós_expandidos  $\leftarrow$  nós_expandidos + 1;
3   if profundidade = 0 ou estado é terminal then
4     | return heurística(tabuleiro, jogador_max);
5   end
6   jogadas  $\leftarrow$  jogadas_válidas(tabuleiro, jogador_atual);
7   if jogadas =  $\emptyset$ ; // passe forçado
8     then
9       | return AlphaBeta(tabuleiro, oponente(jogador_atual), jogador_max, profundidade - 1,  $\alpha$ ,  $\beta$ ,
10        | heurística);
11   end
12   if jogador_atual = jogador_max then
13     | valor  $\leftarrow$   $-\infty$ ;
14     | foreach (jogada, filho)  $\in$  ordenar_filhos(jogadas) do
15       | valor  $\leftarrow$  max(valor, AlphaBeta(filho, oponente(jogador_atual), jogador_max,
16       | profundidade - 1,  $\alpha$ ,  $\beta$ , heurística));
17       |  $\alpha \leftarrow$  max( $\alpha$ , valor);
18       | if  $\beta \leq \alpha$  then
19         | | break; // poda beta
20       | end
21     | end
22   else
23     | valor  $\leftarrow$   $+\infty$ ;
24     | foreach (jogada, filho)  $\in$  ordenar_filhos(jogadas) do
25       | valor  $\leftarrow$  min(valor, AlphaBeta(filho, oponente(jogador_atual), jogador_max,
26       | profundidade - 1,  $\alpha$ ,  $\beta$ , heurística));
27       |  $\beta \leftarrow$  min( $\beta$ , valor);
28       | if  $\beta \leq \alpha$  then
29         | | break; // poda alfa
30       | end
31     | end
32   end
33   return valor;
34 fim
```

O impacto empírico da poda pode ser quantificado comparando o número médio de nós expandidos por jogada entre Minimax e Alpha-Beta sob a mesma heurística e orçamento de tempo (0,5s), conforme a Tabela 1.

Heurística	Minimax (nós)	Alpha-Beta (nós)	Redução
Corner	3724,46	1630,73	56,2%
Frontier	1917,55	1327,70	30,8%
Mobility	959,47	850,83	11,3%

Table 1: Redução no número médio de nós expandidos por jogada com Alpha-Beta em relação ao Minimax.

A heurística *corner* apresenta a maior taxa de poda (56,2%) porque a avaliação de cantos produz escores mais discriminativos na ordenação de jogadas, gerando cortes mais precoces. Já a heurística de *mobility* apresenta menor redução (11,3%) pois a contagem de movimentos válidos oscila rapidamente entre estados vizinhos, produzindo uma ordenação menos eficaz.

3.3 Iterative Deepening e Limite de Tempo

Para operar sob o orçamento computacional fixo de 0,5s por jogada, ambos os algoritmos utilizam *Iterative Deepening* combinado com controle temporal. A ideia é executar buscas com profundidade crescente ($d = 1, 2, 3, \dots$), mantendo sempre o melhor lance encontrado na iteração anterior. Caso o tempo se esgote durante uma iteração, ela é abortada via exceção `TimeoutError` e o lance da última iteração completa é retornado.

Algoritmo 4: Iterative Deepening com Limite de Tempo

```
1 Função Busca.Iterativa(tabuleiro, jogador, tempo_limite, profundidade_max, heurística):
2   deadline  $\leftarrow$  tempo_atual() + tempo_limite;
3   jogadas  $\leftarrow$  jogadas_válidas(tabuleiro, jogador);
4   melhor_lance  $\leftarrow$  primeiro(jogadas) ;                               // fallback seguro
5   for d  $\leftarrow$  1 to profundidade_max do
6     if tempo_atual()  $\geq$  deadline then
7       break;
8     end
9     candidato  $\leftarrow$  Busca(tabuleiro, jogador, d, deadline, heurística);
10    if candidato  $\neq$  nulo then
11      melhor_lance  $\leftarrow$  candidato;
12    end
13    TimeoutError break;
14  end
15  return melhor_lance;
16 fim
```

Essa abordagem garante comportamento *anytime*: o agente sempre possui um lance válido para retornar, independentemente de quando o tempo se esgota. A verificação de *deadline* é realizada em cada nó expandido da árvore, assegurando interrupção rápida. No torneio, utilizou-se *tempo_limite* = 0,5s e *profundidade_max* = 64 (efetivamente ilimitada, sendo o tempo o verdadeiro limitante).

3.4 Ordenação de Jogadas (*Move Ordering*)

A eficácia do Alpha-Beta depende criticamente da ordem em que os nós filhos são explorados. Quando os melhores lances são avaliados primeiro, as podas ocorrem mais cedo e a busca atinge maior profundidade dentro do mesmo orçamento de tempo.

A implementação utiliza uma função de ordenação que, para cada lance legal, simula o movimento resultante e avalia o estado filho com uma heurística rápida (*corner_heuristic*, controle de cantos). Os filhos são então ordenados em ordem decrescente de *escore* para o jogador MAX e crescente para o jogador MIN.

Algoritmo 5: Ordenação de Filhos

```
1 Função Ordenar_Filhos(tabuleiro, jogador_atual, jogador_max, heurística_rápida, turno_max):
2   filhos  $\leftarrow$  [];
3   foreach jogada  $\in$  jogadas_válidas(tabuleiro, jogador_atual) do
4     estado_filho  $\leftarrow$  simular_jogada(tabuleiro, jogada, jogador_atual);
5     escore  $\leftarrow$  heurística_rápida(estado_filho, jogador_max);
6     Adicionar (escore, jogada, estado_filho) a filhos;
7   end
8   if turno_max then
9     Ordenar filhos por escore decrescente;
10  else
11    Ordenar filhos por escore crescente;
12  end
13  return filhos;
14 fim
```

4 Funções Heurísticas

Implementou-se três heurísticas com diferentes estratégias para resolver o jogo Othello para cada algoritmo:

4.1 Mobilidade

A heurística de mobilidade quantifica o grau de liberdade de movimentação do jogador e busca maximizar a diferença entre as ações válidas do agente em relação às ações válidas do oponente. A lógica intrínseca é sufocar o oponente deixando-o sem boas jogadas. O algoritmo computa o total de lances válidos disponíveis para o agente atual

($|A(p)|$) e subtrai as ações permitidas para o adversário ($|A(\neg p)|$). Nesse caso, utiliza-se a função de utilidade $U_{mobility}$ descrita na seção 2.1

4.2 Controle de Cantos

No Othello, discos posicionados nos cantos do tabuleiro - (1, 1), (1, 8), (8, 1), (8, 8) - não podem ser flanqueados. A heurística do controle de cantos calcula a diferença do controle entre os cantos de acordo com a função U_{corner} descrita na seção 2.1. Esta métrica bonifica estados que capturam os cantos e domina as bordas.

4.3 Discos de Fronteira

Penaliza peças do agente que estão adjacentes a pelo menos um espaço completamente vazio no tabuleiro. Discos de fronteira correm grande risco de serem facilmente revertidos nas jogadas subsequentes pelo inimigo, representando uma vulnerabilidade do jogador. Nessa heurística, busca-se minimizar o número de discos de fronteira contabilizando as casas adjacentes livres, forçando o algoritmo a selecionar ações que minimizem o número de casas adjacentes expostas. A utilidade $U_{frontier}$ está descrita na seção 2.1.

5 Avaliação Experimental

A fim de realizar a avaliação dos algoritmos, realizou-se um torneio automatizado em que as combinações algoritmo+heurística enfrentam todas as demais permutações sob as mesmas regras e orçamento computacional. Cada par ordenado de agentes disputou múltiplas partidas, alternando quem joga de pretas e brancas, totalizando 1568 jogos.

Métrica	Valor
Total de jogos	1568
Vitórias (Pretas)	678
Vitórias (Brancas)	848
Empates	42
Tempo máximo por agente (s)	0,5
Profundidade máxima (nós)	64

Table 2: Resumo geral dos jogos

Observa-se na Tabela 2 que as peças brancas venceram mais partidas (848 vs. 678). Esse viés está relacionado à vantagem posicional do segundo jogador no Othello: as brancas reagem ao posicionamento das pretas e frequentemente conseguem melhores posições nos cantos nas fases intermediárias.

5.1 Comparação: Minimax vs Alpha-Beta

A Figura 4 mostra que as taxas de vitória agregadas de Minimax e Alpha-Beta são semelhantes. Isso ocorre porque, dentro do orçamento de 0,5s, ambos alcançam profundidades suficientes para tomar decisões competitivas. Entretanto, Alpha-Beta atinge essas decisões de forma substancialmente mais eficiente.

A Tabela 3 detalha a comparação por heurística. Alpha-Beta expande consistentemente menos nós e alcança maior profundidade média em todos os cenários, com destaque para a heurística de cantos, onde a redução de nós é de 56,2% e o ganho de profundidade é de +1,12 níveis.

Heurística	Nós (Minimax)	Nós (A-B)	Prof. (Minimax)	Prof. (A-B)
Corner	3724,46	1630,73	4,63	5,75
Frontier	1917,55	1327,70	4,31	4,92
Mobility	959,47	850,83	3,78	4,42

Table 3: Nós expandidos e profundidade média por jogada: Minimax vs. Alpha-Beta, por heurística.

Em média, Alpha-Beta alcança aproximadamente 1 nível a mais de profundidade que o Minimax puro sob o mesmo limite de tempo, o que confirma a importância da poda para explorar a árvore de forma mais profunda sem custo adicional de tempo.

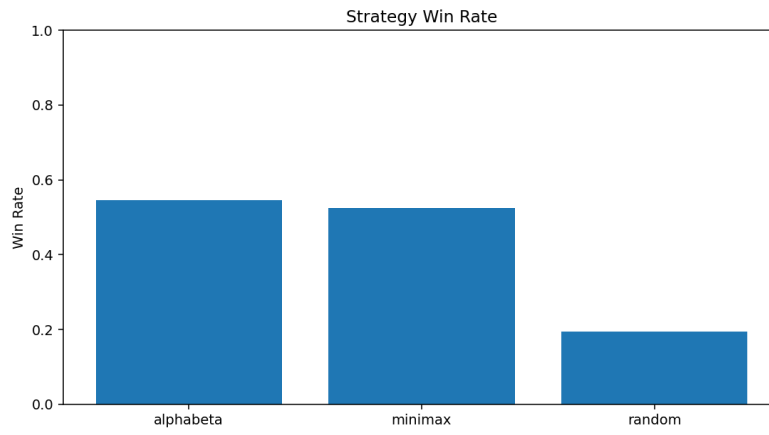


Figure 4: Taxas de vitória por estratégia. O agente *random* tem performance muito inferior às demais estratégias, enquanto Minimax e Alpha-Beta apresentam desempenhos similares.

5.2 Comparação entre Heurísticas

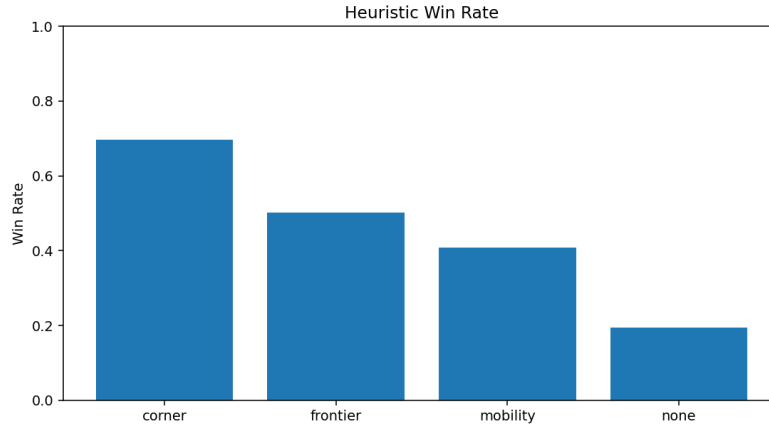


Figure 5: Taxas de vitória por heurística. A heurística de cantos se destaca como a mais eficaz, seguida por mobilidade e fronteira; a ausência de heurística (random) apresenta o pior desempenho.

A Figura 5 apresenta as taxas de vitória agregadas por heurística. A heurística de **cantos** (corner) é claramente superior, com os agentes *alphabeta+corner* e *minimax+corner* atingindo taxas de vitória de 73,9% e 65,4%, respectivamente.

A superioridade da heurística de **cantos** é esperada no Othello: discos posicionados nos cantos são irrevogáveis (não podem ser flanqueados), conferindo vantagem estratégica permanente. A partir dos cantos, é possível estabilizar toda uma borda, acumulando peças que não podem ser viradas.

A heurística de **fronteira** apresenta desempenho intermediário (54,5% para minimax, 45,8% para alpha-beta). Sua estratégia de minimizar discos expostos é defensivamente razoável, mas não captura oportunidades ofensivas como a posse de cantos.

A heurística de **mobilidade** apresenta resultados inferiores (37,7% para minimax, 44,0% para alpha-beta). A contagem de movimentos válidos é uma métrica ruidosa: flutua rapidamente entre estados vizinhos e não reflete diretamente o valor posicional das peças no tabuleiro.

5.3 Comparação entre Agentes

Cada agente é composto por uma estratégia + heurística (usada como função de utilidade). Como *baseline*, usamos um agente que seleciona jogadas aleatoriamente.

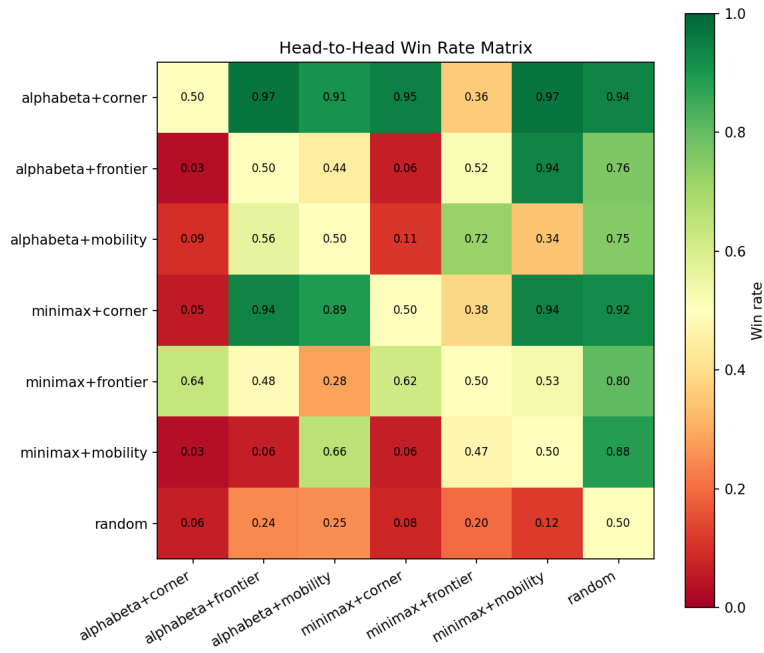


Figure 6: Matriz de taxa de vitória (*head-to-head*) entre todos os pares de agentes. Estratégias informadas superam consistentemente o agente *random* e apresentam desempenho equilibrado entre si, com exceções notáveis.

Agente	Wins	Draws	Losses	Win Rate	Disc Diff	Nodes	Depth	Time (s)
alphabeta+corner	331	53	64	0,739	15,65	1630,73	5,75	0,09493
alphabeta+frontier	205	5	238	0,458	-1,92	1327,70	4,92	0,09553
alphabeta+mobility	197	0	251	0,440	-0,37	850,83	4,42	0,09514
minimax+corner	293	6	149	0,654	9,97	3724,46	4,63	0,09542
minimax+frontier	244	6	198	0,545	1,13	1917,55	4,31	0,09590
minimax+mobility	169	3	276	0,377	-7,10	959,47	3,78	0,09566
random	87	11	350	0,194	-17,35	0,00	0,00	0,00000

Table 4: Desempenho detalhado dos agentes

A Tabela 4 e a Figura 6 revelam que o agente *alphabeta+corner* é o mais forte do torneio, com taxa de vitória de 73,9% e diferença média de discos de +15,65. Todos os agentes com estratégia+heurística superam significativamente o *baseline* aleatório (19,4%), validando a eficácia da busca adversarial.

A diferença média de discos (*disc diff*) correlaciona-se com a taxa de vitória: os agentes de cantos apresentam os maiores valores positivos, enquanto o agente *random* acumula déficit de -17,35 discos por partida. O tempo médio por jogada é de aproximadamente 0,095s para todos os agentes informados, bem dentro do orçamento de 0,5s, confirmando alocação justa de recursos computacionais.

5.4 Profundidade Média e Nós Expandidos

A análise das colunas de profundidade e nós expandidos da Tabela 4 revela a relação entre eficiência algorítmica e qualidade da busca.

O agente *alphabeta+corner* atinge a maior profundidade média (5,75) enquanto expande um número moderado de nós (1630,73 por jogada). Em contraste, *minimax+corner* expande o maior número de nós do torneio (3724,46) mas alcança profundidade inferior (4,63). Essa diferença de 1,12 níveis é inteiramente atribuível às podas do Alpha-Beta.

A profundidade alcançada é limitada pelo produto entre fator de ramificação e orçamento temporal. Com fator de ramificação médio $b \approx 10$ no Othello e limite de 0,5s, cada nível adicional requer $\sim 10\times$ mais processamento. As podas do Alpha-Beta efetivamente reduzem o fator de ramificação efetivo, permitindo explorar mais níveis no mesmo intervalo de tempo.

Outro dado relevante é que a heurística de cantos, além de produzir melhores avaliações, também gera melhor ordenação de jogadas (pois é usada como heurística rápida de ordenação), criando um ciclo virtuoso: melhor

ordenação → mais podas → maior profundidade → decisões mais informadas.

6 Discussão e Conclusões

6.1 Gargalos Computacionais e Limitações Observadas

A instrumentação demonstrou que o principal gargalo temporal concentra-se na *cópia da representação matricial de estados* e na lógica de validação do motor de jogo em Python puro. A cada nó expandido na árvore de busca, a função `simulate_move` clona o tabuleiro inteiro (8×8 inteiros via *list comprehension*) e, em seguida, `valid_moves` percorre todas as 64 células testando 8 direções por célula para determinar as jogadas legais.

Essas operações, embora algoritmicamente corretas, são custosas em Python interpretado. Uma alternativa amplamente utilizada em motores de Othello de alto desempenho é a representação por *bitboards*: dois inteiros de 64 bits (um por jogador) codificam o tabuleiro inteiro, e operações como geração de movimentos e aplicação de jogadas reduzem-se a operações bit-a-bit com complexidade $O(1)$.

Apesar dessas limitações, os agentes alcançam profundidades práticas relevantes (3,78 a 5,75 níveis) dentro do orçamento de 0,5s, demonstrando que a implementação é funcional para os objetivos do trabalho. A profundidade limitada é parcialmente compensada pela qualidade das heurísticas, especialmente a de cantos.

6.2 Relação entre Profundidade de Busca e Qualidade de Jogo

Os dados experimentais permitem analisar a relação entre profundidade alcançada e desempenho competitivo dos agentes. A Tabela 5 sintetiza essa comparação.

Heurística	Prof. Minimax	Prof. Alpha-Beta	Δ Profundidade
Corner	4,63	5,75	+1,12
Frontier	4,31	4,92	+0,61
Mobility	3,78	4,42	+0,64

Table 5: Profundidade média por jogada: Minimax vs. Alpha-Beta.

Alpha-Beta atinge consistentemente maior profundidade que o Minimax sob o mesmo orçamento temporal, com ganho médio de +0,79 níveis. Entretanto, profundidade por si só não garante melhor desempenho. O agente *minimax+corner*, com profundidade média de 4,63, obtém taxa de vitória de 65,4% — superior ao *alphabeta+frontier* (profundidade 4,92, taxa de vitória 45,8%). Isso demonstra que **a qualidade da heurística domina o efeito da profundidade adicional**: um nível extra de busca com uma heurística ruidosa vale menos que uma avaliação precisa em profundidade ligeiramente menor.

Os retornos decrescentes observados explicam-se pelo fator de ramificação do Othello ($b \approx 10$): cada nível adicional multiplica o espaço de busca por $\sim 10\times$, de modo que os ganhos da poda Alpha-Beta são rapidamente consumidos. A combinação ideal, confirmada pelos resultados, é um algoritmo de poda eficiente (Alpha-Beta) aliado a uma heurística com forte correlação ao valor real do estado (controle de cantos).

6.3 Conclusão

Este trabalho implementou e avaliou agentes de busca adversarial para o jogo Othello, utilizando os algoritmos Minimax e Alpha-Beta Pruning com *Iterative Deepening*, ordenação de jogadas e três funções heurísticas distintas (mobilidade, controle de cantos e discos de fronteira).

O torneio automatizado com 1568 partidas demonstrou que:

1. O agente *alphabeta+corner* é o mais forte, com taxa de vitória de 73,9% e diferença média de +15,65 discos por partida.
2. Alpha-Beta reduz o número de nós expandidos em até 56,2% em relação ao Minimax puro, permitindo buscas ~ 1 nível mais profundas no mesmo orçamento de tempo.
3. A qualidade da heurística é o fator dominante no desempenho: a heurística de cantos supera as demais independentemente do algoritmo de busca utilizado.
4. A ordenação de jogadas baseada na heurística de cantos potencializa as podas do Alpha-Beta, criando um ciclo virtuoso entre ordenação, poda e profundidade.